



Sri Lanka Institute of Information Technology

Ocean Sense : AN IOT-BASED HYBRID SYSTEM FOR REAL-TIME DETECTION AND TRACKING OF MARINE DEBRIS

Design Document (DD)

Project ID: 25-26J-002

Submitted by:

Registration #	Student Name
IT22169730	D M T N DISSANAYAKA

Submitted to:

Supervisor - Ms. Dinithi Pandithage

Date of submission : 1/11/2026

Table of Contents

Table of Contents	i
1. Introduction.....	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 References	2
1.5 Overview	4
2. Overall Description.....	5
2.1 Product Perspective	5
2.2 Product Functions.....	7
2.3 User Characteristics.....	8
2.4 Constraints.....	9
2.5 Assumptions and dependencies	9
2.6 Apportioning of requirements	9
3. Specific Requirements	10
3.1 External Interface Requirements	10
3.2 Architectural Design.....	11
3.3 <i>Performance Requirements</i>	13
3.4 Design Constraints.....	14
3.5 Software System Attributes	14
3.6 Other Requirements.....	15
4. Supporting Information	16
4.1 Table of Contents and Indexing.....	16
Table of Contents.....	16
4.2 Appendix	16

1. Introduction

1.1 Purpose

The document serves as a blueprint for the stakeholders to obtain information and understanding about crucial functional and non-functional requirements of the project. Development, design procedures and decisions which are crucial for the management and the design team is also outlined in the document. This document will act as primary reference for implementation, testing and maintenance requirements of the AI Infrastructure Management System of Ocean Sense.

1.2 Scope

*This document covers the architectural design and detailed requirements for the **OceanSense** platform, specifically focusing on the backend infrastructure and the Machine Learning Model Management System (MMS).*

Scope of this document includes the following,

- **Infrastructure Management:** *Design specifications for the Datastore module responsible for ingesting data from IoT devices and managing metadata tagging (including preliminary AI-based tagging).*
- **Model Management:** *The architecture for the Model Management System (MMS), which handles the importation of data from the Datastore and triggers automated training pipelines.*
- **ML Capabilities:** *The design supports specific use cases for Ocean Plastic/Waste Detection and Harmful Algal Bloom (HAB) detection.*
- **System Boundaries:** *This document focuses on the backend logic, API specifications, and data flow required to achieve the current project milestone (functional backend with emphasis on five trained models), while acknowledging the frontend as a consumer of these APIs.*

1.3 Definitions, Acronyms, and Abbreviations

<i>Term</i>	<i>Definition</i>
<i>MMS (Model Management System)</i>	<i>The subsystem responsible for managing ML models, versioning, and triggering training workflows.</i>
<i>Data Store</i>	<i>Central repository for raw images and metadata collection endpoint from IoT end devices.</i>
<i>IoT (Internet of Things)</i>	<i>The Edge devices that are intended to be deployed (in the marine context for this research demonstration)</i>
<i>SAM (Segment Anything Model)</i>	<i>Used for zero shot segmentation tasks to assist data labeling process.</i>
<i>FastAPI</i>	<i>The web frame work used for building the high performance backend APIs</i>
<i>Min IO</i>	<i>High performance object storage for managing large datasets for images</i>
<i>Celery</i>	<i>Distributed task queue used to handle async background tasks like model training and data processing</i>

1.4 References

1.4.1 Ocean Plastic & Debris Detection (Deep Learning)

[1] Z. Jian, Y. Hu, G. Chen, Z. Wang, and Z. Feng, "An Efficient Deep-Sea Debris Detection Method Using Deep Neural Networks," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 14, pp. 12348–12360, Nov. 2021.

[2] K. S. Babu, T. R. G. Kumar, and S. Kumar, "Detection of Underwater Marine Plastic Debris Using an Augmented Low Sample Size Dataset for Machine Vision System: A Deep Transfer

Learning Approach," in *2021 IEEE International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, Bangalore, India, 2021, pp. 1–6.

[3] J. Fulton, J. Hong, M. Islam, and J. Sattar, "Robotic Detection of Marine Litter Using Deep Learning for Underwater Environmental Monitoring," in *2019 International Conference on Robotics and Automation (ICRA)*, Montreal, QC, Canada, 2019, pp. 5752–5758.

1.4.2 Harmful Algae Bloom (HAB) Detection

[4] P. R. Hill, A. Kumar, M. Temimi, and D. R. Bull, "HAB Detection in Turbid Waters Using a Deep Convolutional Neural Network," *IEEE Geoscience and Remote Sensing Letters*, vol. 17, no. 12, pp. 2060–2064, Dec. 2020.

[5] J. Pullen, A. L. Gordon, and J. Sprintall, "A Deep Learning Paradigm for Detection of Harmful Algal Blooms," in *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, Santa Rosa, CA, USA, 2017, pp. 1–9.

[6] T. Gokul, K. Deepa, and A. Rajan, "Algal Bloom Detection in Marine Environment Using Machine Learning Techniques," in *2019 IEEE International Conference on Intelligent Techniques in Control, Optimization and Signal Processing (INCOS)*, Tamilnadu, India, 2019, pp. 1–4.

1.4.3 Core Technologies

[7] A. Kirillov et al., "Segment Anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Paris, France, 2023, pp. 4015–4026.

[8] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>. [Accessed: Jan. 10, 2025].

[9] S. Ramirez, "FastAPI: High performance, easy to learn, fast to code, ready for production," 2018. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: Jan. 10, 2025].

1.4.4 IoT & System Architecture

[10] M. A. Al-Garadi, A. Mohamed, A. Al-Ali, X. Du, I. Ali, and M. Guizani, "A Survey of Machine and Deep Learning Methods for Internet of Things (IoT) Security," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1646–1685, 3rd Quart., 2020.

1.5 Overview

*The **OceanSense** is an AI-driven workflow management platform designed to automate the detection and analysis of marine hazards, specifically focusing on ocean plastic/waste and harmful algal blooms. The system's primary goal is to bridge the gap between raw data collection and actionable intelligence by integrating IoT infrastructure management with a robust Machine Learning Model Management System (MMS).*

The system is designed to provide the following critical features:

- 1. Data Ingestion & Observation*

Aggregating raw imagery and sensor data from IoT devices in to a central datastore, where preliminary AI assisted enriching (utilizing tools like Segment Anything Model or a Large Language model) of the metadata.

- 2. Model Management & Training*

Providing an environment for dataset curation and automated training pipelines for object detection models.

- 3. Continuous Improvement*

Facilitation of a feedback loop where new data is used to improve accuracy of each model for each context over time, supporting intelligent and personalized use of Machine Learning models for their relevant context for maximum efficiency.

2. Overall Description

2.1 Product Perspective

OceanSense differs from standard passive IoT data loggers or generic image repositories by integrating an active Machine Learning feedback loop directly into the infrastructure management. While traditional systems may simply store collected data for manual review, OceanSense is designed to autonomously refine its detection capabilities over time. It compares favorably to general-purpose ML ops platforms by offering specialized workflows tailored specifically for marine hazard detection (ex: plastics and harmful algae), reducing the configuration burden on environmental scientists.

2.1.1 System Interfaces

The system backend interacts with the host operating system primarily through containerized environments (Docker), requiring a Linux-based kernel for optimal performance of the ML workloads. It interfaces with the file system for temporary storage during image processing before offloading to object storage.

2.1.2 User Interface

The user interface is designed around a web-based Dashboard built with React, focusing on clarity and data visualization.

The UI follows a “monitor-then-act” principle. The primary view provides a real-time feed of ingested IoT data. A secondary "Model Management" view allows users to select datasets and trigger training pipelines without interacting with command-line tools.

2.1.3 Hardware Interfaces

- **IoT Devices:** *The system accepts data input from deployed edge devices (e.g., cameras and environmental sensors) capable of capturing and transmitting high-resolution imagery of the water surface.*

- **Training Hardware:** *The Model Management System requires access to varying grades of GPUs to execute the training of YOLO and SAM models from consumer GPUs to Server grade GPUs.*

2.1.4 Software Interfaces

OceanSense relies on several critical software subsystems to function:

- *Object Storage (MinIO): Interfaces with MinIO for S3-compatible storage of large raw image datasets and trained model artifacts.*
- *Database (PostgreSQL): Interfaces with PostgreSQL to store relational data, including metadata tags, user profiles, and training logs.*
- *Task Queue (Redis/Celery): Interfaces with Redis and Celery to offload long-running tasks such as image preprocessing and model training from the main application thread.*

2.1.5 Communication Interface

- *REST API: The primary communication method between the Frontend and Backend is via HTTP/HTTPS using RESTful endpoints provided by the FastAPI framework.*
- *Internal Messaging: Components communicate asynchronously via a message broker (Redis) to handle job distribution.*

2.1.6 Memory Constraints

- *Operational RAM : The backend services (API, Database, MinIO) require moderate RAM (estimated 4GB-8GB) for standard operation per user*
- *ML Workloads: The training pipelines are memory-intensive. The system assumes availability of significant GPU VRAM (minimum 8GB recommended) and system RAM (16GB+) during active model training phases to handle batch processing of images.*

2.1.7 Operations

The System supports 2 primary modes of operation:

- *Autonomous Ingestion:* Continuous background operation where the system listens for and ingests data from IoT sources without user intervention.
- *Interactive Management:* User-initiated operations for curating datasets, annotating images via the UI, and manually triggering the "Train Model" workflow.

2.1.8 Site adaptation requirements

The system is designed to be deployment-agnostic via Docker containers. However, the physical site hosting the backend must have network connectivity to the IoT devices (or their data uplink) and must provide the necessary hardware drivers (e.g., NVIDIA drivers) for the ML components to function.

2.2 Product Functions

< The OceanSense platform performs five major functions to facilitate the end-to-end lifecycle of marine data analysis.

2.2.1 IoT Data Ingestion & Storage

The system accepts incoming data streams (images and sensor readings) from authenticated IoT devices (cameras, buoys, drones). It validates the data structure and stores raw assets in an S3-compatible object store (MinIO), while indexing structured metadata in a relational database.

2.2.2 Metadata Enrichment

Upon ingestion, the system applies a "preliminary observation" layer. This involves utilizing Large Language Models (LLMs) like GPT-4 Vision or Claude to analyze raw images and automatically generate descriptive tags (e.g., "plastic bottle," "algae bloom," "clear water"). This enriched metadata facilitates efficient querying and dataset creation.

2.2.3 Model Management System

The MMS serves as a central registry for Machine Learning models. It allows users to register new model architectures (like, YOLOv12, EfficientNet), version control models to track changes over time, manage model states (Created, Training, Trained, Deployed, Archived).

2.2.4 Automated Training Orchestration

The system provides a workflow to "intelligently import" data from the Datastore into the training environment. Users can trigger training pipelines that automatically preprocess data, train specific

models (e.g., for Ocean Plastic Detection), and log performance metrics (Precision, Recall, F1-Score) using MLflow.

2.2.4.1 Continuous Improvement

The platform supports a feedback mechanism where new field data is used to retrain and refine models. It enables the comparison of model versions to ensure that detection capabilities for specific hazards (like harmful algae) improve with each iteration.

2.3 User Characteristics

The system is designed for three distinct user categories with varying technical expertise:

2.3.1 Marine Data Scientists (Primary Users):

- **Expertise:** High familiarity with ML concepts (training, validation, overfitting) but may lack deep DevOps knowledge.
- **Role:** They utilize the Model Management System to curate datasets, configure training parameters, and evaluate model performance.

2.3.2 System Administrators:

- **Expertise:** Proficient in Linux, Docker, and network security.
- **Role:** Responsible for deploying the backend infrastructure, managing API keys, ensuring GPU driver compatibility, and monitoring system health.

2.3.3 Environmental Analysts:

- **Expertise:** Domain experts in marine biology or environmental science; non-technical regarding code.
- **Role:** They consume the outputs (detected anomalies, pollution reports) via the Dashboard to make informed decisions regarding marine safety.

2.4 Constraints

- *Hardware Constraints:* The training pipeline is strictly dependent on the availability of NVIDIA GPUs with CUDA support. The host system must have sufficient VRAM (minimum 8GB recommended) to handle batch processing of high-resolution aerial or underwater images.
- *Network Bandwidth:* The system assumes a stable internet connection for the backend to ingest data from IoT devices and to communicate with external LLM APIs (OpenAI/Anthropic) for metadata enrichment.
- *Data Privacy & Compliance:* While focused on marine debris, images may inadvertently capture vessels or persons. The system must process data in compliance with general data privacy standards, ensuring no personally identifiable information (PII) is permanently logged without consent.
- *Containerization:* The entire backend solution is constrained to run within Docker containers to ensure consistency across development and production environments.

2.5 Assumptions and dependencies

- *Third-Party API Availability:* The "Intelligent Tagging" feature assumes uninterrupted access to external LLM services (OpenAI or Anthropic).
- *IoT Standardization:* It is assumed that all edge devices transmit data in standard formats (JPEG/PNG for images, JSON for telemetry) compliant with the API's ingestion schema.
- *Database Persistence:* It is assumed that the PostgreSQL and MinIO storage volumes are mounted on persistent disk storage to prevent data loss during container restarts.

2.6 Apportioning of requirements

The following features are considered outside the scope of this specific document version (v1.0) and are deferred to future releases:

- *Real-time Edge Inference:* Deploying trained models directly onto the IoT devices (Edge AI) is planned for Phase 2; currently, all inference occurs on the backend server.
- *Kubernetes Orchestration:* While Docker Compose is used for the current deployment, full Kubernetes (K8s) auto-scaling capabilities are deferred.

- *Mobile Application: A native mobile interface for field alerts is deferred; the current interface is a responsive web dashboard.*
- *Federated Learning: Collaborative training across decentralized devices without sharing raw data is a future research capability.*

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface

The system's primary user interface is a web-based Dashboard. The UI adheres to Material Design principles for consistency and responsiveness across desktop and tablet devices.

- *Navigation Bar: Persistent access to "Live Feed," "Data Explorer," "Model Registry," and "Settings."*
- *Geographical Information System: Data classified under classifications based on geographical information represented as a heat map of aggregate data for specific use case. (Auto generation of specific use cases may be available with the help of an LLM style interface)*
- *Live Feed View: Displays a grid of recently ingested images from IoT devices with overlay tags (e.g., "Plastic: 85%").*
- *Model Registry View: A table listing available models, their versions, current status (Training/Ready), and performance metrics (F1-Score).*
- *Training Control Panel: A form interface to select a dataset time range, choose a base model architecture (e.g., YOLOv8), and trigger a training job*

3.1.2 Hardware Interface

- *IoT Data Ingestion: The backend exposes a specific endpoint (/api/data/ingest) configured to accept multipart/form-data from varying hardware sources, including Raspberry Pi-based cameras and industrial buoy sensors.*

- *GPU Passthrough: The design requires the Docker container runtime to utilize the NVIDIA Container Toolkit, allowing the application containers to directly access the host's GPU hardware for CUDA-accelerated operations.*

3.1.3 Software Interfaces

- *Operating System: The system is designed for Linux-based host environments (Ubuntu 20.04 LTS or later recommended) to support Docker and NVIDIA drivers.*
- *Database System: PostgreSQL (v15) is used for relational data storage. The application interfaces with it via SQLAlchemy ORM.*
- *Object Storage: MinIO is used as the software interface for blob storage, mimicking AWS S3 API protocols.*
- *AI Services: The system interfaces with OpenAI API (GPT-4 Vision) and Anthropic API (Claude 3) for optional metadata enrichment tasks.*

3.1.4 Communication Interfaces

- *Protocol: HTTP/1.1 and HTTP/2 over TCP/IP.*
- *Data Format: JSON (application/json) for control messages; binary for image payloads.*
- *Ports:*
 - 8000: Backend API (FastAPI)
 - 5432: PostgreSQL Database
 - 9000: MinIO Console
 - 6379: Redis

3.2 Architectural Design

3.2.1 High Level Architectural Design

The OceanSense platform follows a Microservices-based Architecture containerized via Docker. The system is decomposed into loosely coupled services that communicate via REST APIs and an asynchronous task queue.

Client Layer: *Includes IoT devices pushing data and Web Dashboards pulling analytics.*

API Gateway: The FastAPI backend serves as the single entry point, routing requests to internal modules.

Service Layer:

- *Datastore Service:* Handles ingestion and storage logic.
- *Model Registry:* Manages model metadata and versioning.
- *Training Orchestrator:* Manages the lifecycle of ML training jobs.

Data Layer: Consists of PostgreSQL (metadata) and MinIO (raw assets).

Async Worker Layer: Celery workers backed by Redis handle resource-intensive tasks (training, image processing) to keep the API responsive

3.2.2 Hardware and software requirements with justification

<i>Requirement</i>	<i>Specification</i>	<i>Justification</i>
<i>Server CPU</i>	<i>Quad-Core</i>	<i>Required to handle concurrent API requests and background worker orchestration.</i>
<i>Server RAM</i>	<i>16 GB</i>	<i>8 GB dedicated for active ML model training workflow, 8GB for OS/DB/API services</i>
<i>GPU</i>	<i>VRAM 8GB</i>	<i>Deep learning training (YOLO/ResNet) requires CUDA cores for feasible training times. CPU training is too slow.</i>
<i>Storage</i>	<i>500 GB (SSD)</i>	<i>High I/O throughput is needed for reading thousands of image files during training epochs.</i>
<i>Software</i>	<i>Docker</i>	<i>Ensures the application runs identically on the developer machine and the deployment server (eliminates "works on my machine" issues).</i>
<i>OS</i>	<i>Linux</i>	<i>Native support for NVIDIA container toolkit and superior file system performance for large datasets.</i>

3.2.3 Risk Mitigation Plan

Risk	Impact	Mitigation Strategy
Data Loss	High	Frequent backups of PostgreSQL dump and MinIO volume sync to cold storage. Use RAID configuration for disk failure.
Training Failure	Medium-High	Implement "Checkpointing" in training loops. If a job crashes, it resumes from the last saved epoch rather than starting over.
API rate limit	Low	If external LLM APIs (OpenAI) fail or throttle, the system falls back to a locally hosted, lighter model (e.g., BLIP-2) or skips the enrichment step without halting ingestion.
Network Outage	High	IoT devices should implement local buffering (store-and-forward) to upload data once connectivity is restored.

3.2.4 Cost Benefits Analysis

3.2.4.1 Proposed Solution (Custom ML Platform):

- *Cost: High initial development effort; hardware purchase (GPU Server ~\$2,000).*
- *Benefit: No recurring licensing fees per user; full data sovereignty; customized models specifically for "ocean plastics" yield higher accuracy than generic cloud vision APIs.*

3.2.4.2 Alternative (Cloud SAAS):

- *Cost: Pay-per-image analysis (approx. \$1 per 1000 images). For continuous IoT streams (10k images/day), costs escalate to ~\$300/month indefinitely.*
- *Benefit: Zero maintenance; instant scalability.*

The proposed self-hosted solution breaks even within 6-8 months and offers superior privacy and model customization for this specific scientific domain.

3.3 Performance Requirements

3.3.1 Speed and Latency

- *API Response Time: The backend API, built on FastAPI, is required to respond to standard HTTP GET requests (e.g., retrieving model status) within 200 milliseconds under normal load.*

- **Ingestion Throughput:** *The system must handle the concurrent ingestion of at least 100 data entries per minute from IoT devices without dropping packets, utilizing background worker queues (Celery/Redis) to decouple ingestion from processing.*
- **Training Time:** *For the "Ocean Plastic Waste Detector" (ResNet50 based), the system must complete a fine-tuning epoch on a dataset of 1,000 images within 15 minutes when utilizing the recommended NVIDIA GPU hardware.*

3.3.2 Resource Usage

- **Static Memory:** *The baseline container stack (PostgreSQL, Redis, API, MinIO) requires approximately 4GB RAM to remain idle.*
- **Dynamic Memory:** *During active model training, the system is designed to consume up to 90% of available GPU VRAM and utilizes system RAM for data pre-fetching. The application must gracefully fail (raise ResourceExhausted error) rather than crashing the OS if limits are exceeded.*

3.4 Design Constraints

- **Language & Framework:** *The backend logic is constrained to Python 3.10+ to leverage modern type hinting and the specific asynchronous features of FastAPI.*
- **Containerization Standard:** *All deliverables must be packaged as Docker images to ensure the environment is identical between the university lab computers and the deployment server.*
- **Stateless API:** *The REST API design must remain stateless; no client session data should be stored in the application memory (RAM), ensuring that the API container can be restarted without logging out users or losing state (state is delegated to Redis/PostgreSQL).*
- **Library Dependencies:** *The ML components are restricted to PyTorch and scikit-learn frameworks as per the current syllabus requirements and available documentation.*

3.5 Software System Attributes

3.5.1 Reliability

The system employs persistence isolation; the database and object storage run in separate containers with mounted volumes. If the application container crashes due to a coding error, the data remains intact. The restart: always policy in Docker Compose ensures services automatically attempt to recover after a crash.

3.5.2 Availability

The system is designed for 99.5% availability during business hours. Maintenance windows (for model updates) are handled by deploying new containers before taking down old ones (rolling updates), though full zero-downtime deployment is a future goal.

3.5.3 Security

- **Input Validation:** *All incoming data payloads are strictly validated against Pydantic models to prevent malformed data or injection attacks.*
- **Secret Management:** *Sensitive credentials (database passwords, OpenAI API keys) are never hardcoded; they are injected solely via environment variables (.env file) at runtime.*
- **Access Control:** *Access to the train endpoints is restricted to authenticated users to prevent unauthorized consumption of expensive GPU resources.*

3.5.4 Maintainability

The codebase is separated into distinct directories for datastore, models, and training, allowing different developers to work on ingestion logic and ML algorithms simultaneously without merge conflicts. The use of Python type hints ensures that IDEs can catch type-related errors before runtime, significantly reducing debugging time for future student maintainers.

3.6 Other Requirements

- **Logging :** *All critical system events (Ingestion Success, Training Start, Training Failure) must be logged to stdout in JSON format to be easily readable by log aggregation tools.*
- **Documentation:** *The code must include Python Docstrings for every API endpoint, automatically generating the Swagger UI documentation at /docs.*

4. Supporting Information

4.1 Table of Contents and Indexing

Table of Contents

Table of Contents	i
1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 References	2
1.5 Overview	4
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions	7
2.3 User Characteristics	8
2.4 Constraints	9
2.5 Assumptions and dependencies	9
2.6 Apportioning of requirements	9
3. Specific Requirements	10
3.1 External Interface Requirements	10
3.2 Architectural Design	11
3.3 <i>Performance Requirements</i>	13
3.4 Design Constraints	14
3.5 Software System Attributes	14
3.6 Other Requirements	15
4. Supporting Information	16
4.1 Table of Contents and Indexing	16
Table of Contents	16
4.2 Appendix	16

4.2 Appendix

Appendix A : Specialized Model specifications

The OceanSense platform integrates five distinct Machine Learning models tailored for specific marine environments. The following table details the architecture and purpose of each model currently registered in the system

<i>Model ID</i>	<i>Model Name</i>	<i>Base Architecture</i>	<i>Target Application</i>

<i>ocean_plastic_v1</i>	<i>Ocean Plastic Waste Detector</i>	<i>ResNet50</i>	<i>Identification of floating plastic debris in open water images.</i>
<i>hab_detect_v1</i>	<i>Harmful Algae Bloom Detector</i>	<i>EfficientNet-B0</i>	<i>Analysis of water surface discoloration to detect potential toxic algae blooms.</i>
<i>marine_debris_v1</i>	<i>Marine Debris Classifier</i>	<i>VGG16-BN</i>	<i>Classification of debris types (e.g., fishing nets, metal, wood) for cleanup prioritization.</i>
<i>water_qual_v1</i>	<i>Water Quality Assessor</i>	<i>DenseNet121</i>	<i>Estimation of water turbidity and clarity levels from visual sensors.</i>
<i>coastal_poll_v1</i>	<i>Coastal Pollution Monitor</i>	<i>MobileNetV3-Large</i>	<i>Lightweight detection of pollution accumulation along shorelines (optimized for ocean drone feeds).</i>

Appendix B : Sample Input/Output Formats

The following JSON payloads represent the standard data structures for communicating with the OceanSense API.

B.1 : Data Ingestion Payload [Post /api/data/ingest]

```
{
  "source_id": "cam_buoy_04",
  "timestamp": "2024-03-15T08:30:00Z",
  "location": {
    "lat": 6.9271,
    "lon": 79.8612
  },
  "data_type": "image/jpeg",
  "metadata": {
    "weather": "clear",
    "sea_state": "moderate"
  }
}
```

B.2 : Model Train Request [Post / api/models/train]

```
{
  "model_id": "ocean_plastic_v1",
  "dataset_filter": {
```

```
"start_date": "2024-01-01",  
"end_date": "2024-02-01",  
"tags": ["plastic", "verified"]  
},  
"hyperparameters": {  
  "epochs": 50,  
  "batch_size": 32,  
  "learning_rate": 0.001  
}
```

Appendix C: Deployment & Packaging Instructions

The system is packaged as a multi-container Docker application to ensure consistency across development and production environments.

Prerequisites:

Docker Engine (v24.0+)

NVIDIA Container Toolkit (for GPU support)

Docker Compose

Start-up Command:

To initialize the database, object storage, and API services, execute the following from the project root:

docker-compose up -d --build

Access Points:

- *API Documentation (Swagger UI): <http://localhost:8000/docs>*
- *MinIO Console (Storage): <http://localhost:9001>*
- *MLflow Dashboard (Training Metrics): <http://localhost:5000>*